

PilotFish Attribute Population Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **sticky-note step** for your route. You define one or more name/value pairs — literal text, environment variables, or expressions evaluated against the current transaction — and the processor writes them onto the transaction (or onto a running process) before later stages run.

Attribute Population — what it does.

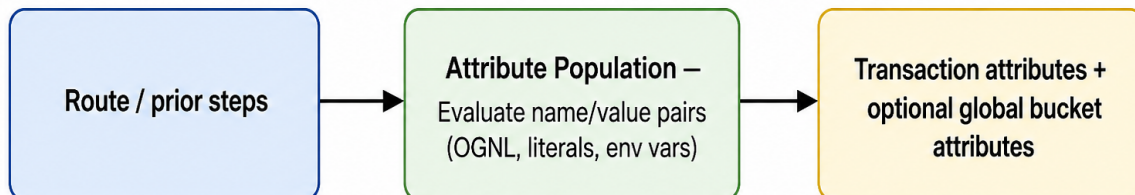


Figure 1: At a glance (plain English)

- **Sets** transaction attributes from a configurable name/value table.
- **Evaluates** names and values through the configuration expression engine (OGNL / enhanced expressions).
- **Optionally** writes to **process-scoped** attributes when orchestrating multi-step flows.

- **Optionally** publishes **global** attributes visible across transactions (bucket + name + value).

Purpose

This document is a **deep dive** into the **Attribute Population** processor (UI type **Transaction Attribute Population**) as shipped in **eip.war.hs.26R1.11**: how it evaluates attribute pairs, chooses transaction vs process scope, sets global bucket attributes, and migrates legacy single-attribute configuration.

Provenance	Value
WAR	eip.war.hs.26R1.11
Module JAR	xcs-modules-core-1.0.3.jar
Decompiler	CFR 0.152
Implementation class	com.pilotfish.eip.modules.internal.TransactionAttributePopulation
Decompiled path	war-pipeline/decompiled/eip-26R1.11/.../TransactionAttributePopulation
Processor type string	Transaction Attribute Population
Description	Populates a single transaction attribute with the results of a general expression, which can be a literal value, environmental variable, or OGNL expression.
Help ID	console.processor.transaction_attribute_population
Categories	ATTRIBUTES, PILOT_FISH

Derived from WAR bytecode (CFR). Narrative follows executable logic in `processData`. The description text predates the multi-row **Attributes to Set** table but remains accurate for each row.

1. Conceptual model

Attribute Population is a **metadata enrichment** step. It does **not** read or modify the transaction data stream.

Transaction in route

|
v

For each row in AttributePairs:

```
    evaluate Name -> ConfigurationManager.getValueFor(...)
    evaluate Value -> ConfigurationManager.getValueFor(...)
    |
    +-- scope = Transaction (default)
    |     -> data.getAttributes().setAttribute(name, value)
    |
    +-- scope = Process
          -> ProcessOrchestrationManager.getOrCreateProcessInstance(data)
              .setAttribute(name, value)
```

```

For each row in GlobalAttributes (Bucket, Name, Value):
    TransactionAttributes.setGlobalAttribute(bucket, name, value)
    |
    v
Same TransactionData returned (attributes updated)

```

2. High-level behavior (processData)

1. Parse attribute pairs

- `TableConfigurationItem.convertUnparsedInputToMetaObject(manager.getStringValue("Attribute", data))`
- Empty table -> loop does nothing (no error).

2. Resolve orchestration scope

- Read `AttributeScope` string; default UI choice is **Transaction**.
- If scope equals **Process** (`AbstractConfiguration.PROCESS`):
 - `ProcessOrchestrationManager.getInstance().getOrCreateProcessInstance(data)`
 - If null -> `EIPEException("Error trying to set process attributes as no process was associated with the transaction. Please set com.pilotfish.eip.process. as a transaction attribute 1st.")`
 - Each pair -> `process.setAttribute(String.valueOf(name), value)` after expression evaluation.
- Else (transaction scope):
 - Each pair -> `data.getAttributes().setAttribute(String.valueOf(name), value)`.

3. Global attributes

- Parse `GlobalAttributes` table via `convertInputToUnparsedTupleMetaObject`.
- For each tuple: evaluate bucket (column 0), name (column 1), value (column 2); call `TransactionAttributes.setGlobalAttribute(bucket, name, value)`.
- Default bucket placeholder in descriptor is **BUCKET**.

4. Return the same `TransactionData` instance.

5. Any exception is wrapped: `EIPEException("Failure in Transaction Attribute Population Processor", cause)`.

Names and values pass through `ConfigurationManager.getValueFor(..., data, manager.getContext())`, so enhanced expressions, transaction attribute references, and environment lookups behave like other PilotFish modules.

3. Configuration reference

Items from `getConfigurationDescriptor()` plus inherited `AbstractProcessor` options.

3.1 Attributes to Set

Property	Value
UI label	Attributes to Set
Tag	<code>AttributePairs</code>
Type	Table (Name / Value columns)
Default	empty
Key repeating	not allowed
Empty keys	not allowed
Tooltip	Add new attributes as attribute name / attribute value pairs.

Nuances:

- Both columns are evaluated at runtime; a value can reference `%PreviousAttribute%`, OGNL, or literals.
- Attribute names are coerced with `String.valueOf(name)` before storage.
- Multiple rows are applied in table order; later rows overwrite the same key if duplicated (last wins).

3.2 Attribute Orchestration Scope

Property	Value
UI label	Attribute Orchestration Scope
Tag	<code>AttributeScope</code>
Type	Multiple choice
Choices	<code>Transaction</code> , <code>Process</code>
Default	<code>Transaction</code>
Tab	Process Orchestration
Tooltip	Scopes for Process Orchestration

Nuances:

- **Process** scope requires an associated process instance (typically via `com.pilotfish.eip.process.ProcessInstance` on the transaction).
- Empty scope string is treated as transaction scope (`PROCESS.equals(scope)` is false).

3.3 Global Attributes

Property	Value
UI label	Global Attributes
Tag	<code>GlobalAttributes</code>
Type	Table (Bucket / Name / Value)
Default bucket row	<code>BUCKET</code>
Key repeating	allowed (same bucket, different names)
Empty keys	not allowed

Property	Value
Tooltip	Sets globally accessible attributes which are available to all transactions

Nuances:

- Uses `TransactionAttributes.setGlobalAttribute` — visible outside the current transaction.
- Bucket, name, and value cells are all expression-evaluated.
- Global writes happen **after** transaction/process pairs, regardless of scope.

3.4 Inherited AbstractProcessor options

Property	Value
UI label	Execute Processor
Tag	<code>ExecuteProcessor</code>
Type	Boolean
Default	<code>true</code>
Tab	Conditional Execution
Tooltip	Transaction data dependent condition may be specified here as enhanced expression. If this expression returns anything other than TRUE (ignore case) - this processor will be skipped.

Nuances:

- When false/skipped, no attributes are written and globals are not updated.

4. Legacy configuration migration (`readConfiguration`)

If XML contains legacy tags `AttributeName` and `Expression` but no `AttributePairs` table, the processor converts them at load time:

```
AttributePairs = createEipPairString(AttributeName, Expression)
```

Routes upgraded from older eiConsole versions may still carry the legacy tags on disk until saved through the current UI.

5. Transaction attributes

5.1 Written by this processor

Any keys defined in **Attributes to Set** (transaction or process scope). The processor does not register fixed attribute names.

5.2 Process attributes (Process scope)

Written to the **ProcessInstance** attribute map, not the transaction map. Downstream orchestration steps read them through the process API.

5.3 Global attributes

Written to the global bucket namespace; keys are (**bucket**, **name**) pairs from the Global Attributes table.

5.4 Runtime-only inputs

Attribute	When needed
<code>com.pilotfish.eip.process.ProcessInstance</code>	Required indirectly when Process scope is selected — without a resolvable process instance the processor throws.

6. Worked examples

6.1 Literal routing label

Name	Value
<code>com.acme.RouteStage</code>	Validated

After execution, `data.getAttributes().getAttribute("com.acme.RouteStage")` returns "Validated". Stream unchanged.

6.2 Copy ISA sender into custom key

Name	Value
<code>com.acme.TradingPartner</code>	<code>%com.pilotfish.EDI.ISA06%</code>

Assumes a prior **EDI Identification** (or equivalent) step populated `com.pilotfish.EDI.ISA06`.

6.3 Global counter bucket

Bucket	Name	Value
<code>com.acme.counters</code>	<code>LastProcessedAt</code>	<code>%CurrentDateTime%</code>

All transactions can read the shared value via global attribute APIs.

7. Known quirks and caveats

1. **Does not touch transaction data** — only attributes (and globals).
2. **Process scope fails fast** without orchestration context — error message explicitly mentions `ProcessInstanceId`.
3. **Description mentions “single” attribute** — UI and code support **multiple** rows via `AttributePairs`.
4. **Global attributes always run** — even when attribute pair table is empty.
5. **Exception wrapping** hides the original type — inspect `getCause()` in logs.
6. **`String.valueOf` on names** — non-string evaluated names become string keys.
7. **No validation of attribute name syntax** — invalid or duplicate naming is the route author’s responsibility.

8. Operational recommendations

Goal	Guidance
Route labeling	Place after identification/listener stages that populate source attributes
Orchestration	Use Process scope only when <code>ProcessInstanceId</code> is guaranteed
Globals	Reserve global buckets for truly cross-transaction state; avoid high-churn keys
Expressions	Test enhanced expressions in eiConsole test mode before production
Naming	Prefer reverse-DNS keys (<code>com.customer.*</code>) to avoid collisions
Conditional skip	Use Execute Processor when attributes are needed only on some branches

9. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
Attribute Population processor	com/pilotfish/eip/modules/internal/TransactionAtt
AbstractProcessor	com/pilotfish/eip/extend/AbstractProcessor.java
Process orchestration	com/pilotfish/eip/process/ProcessOrchestrationMan
Module JAR	war-pipeline/jars/eip-26R1.11/xcs-modules-core-1.

10. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> xcs-modules-core-1.0.3.jar

-> CFR decompile

-> Documents/Processors/26R1.11/PilotFish-Attribute-Population-Reference-26R1.11.(md|pdf)

Deep-dive documentation from WAR decompile – Attribute Population Processor – 26R1.11 – August 2026.